

The MediaTek logo is an orange parallelogram with the word "MEDIATEK" in white, bold, sans-serif capital letters.

MT8676 Hypervisor GNSS User Manual

Version: 1.0
Release date: 2024-11-13

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-13	Neo.Sun	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 GNSS 4

1.1 Overview 4

1.1.1 Brief introduction 4

1.1.2 GNSS Abbreviations 4

1.2 Architecture/Process Overview 4

1.2.1 GNSS Architecture 4

1.3 Configuration/Customization Guideline 5

1.3.1 Fixed Rate Configuration 5

1.3.2 Multi-Satellite Navigation System Configuration 6

1.4 GNSS API 6

1.4.1 Code Path 6

1.4.2 GpsInterface 7

1.4.3 CallBack 7

1.4.4 Demo Code 8

1.5 Frequently Asked Questions/Troubleshooting 9

1.5.1 Issues Related to Logs 9

1.5.2 Testing-related Issues 9

1.5.3 GNSS Path 10

Exhibit 1 Terms and Conditions 11

List of Figures

Figure 1-1. GNSS architecture 5

Figure 1-2. Fixed Rate configuration 6

Figure 1-3. GNSS configuration 6

Figure 1-4. MNLD test 10

List of Tables

Table 1-1. Abbreviations 4

1 GNSS

1.1 Overview

1.1.1 Brief introduction

This section introduces the basic functions of MT8676 Global Navigation Satellite System (GNSS) and solutions to common issues.

1.1.2 GNSS Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
COLD start	With time-assisted information, the end user will not encounter this scenario.
FULL start	Without any auxiliary information, equivalent to the scenario where an end user uses a positioning application for the first time after purchasing a mobile phone.
GNSS	Global Navigation Satellite System
Hot start	With all auxiliary information, the end user's current positioning occurs less than 2 to 4 hours after the last positioning.
NMEA	National Marine Electronics Association. A communication protocol used for data exchange between marine electronic devices, widely applied in GPS/GNSS receiver data output.
TTFF	Time To First Fix. The time required for a navigation device to successfully acquire the first valid positioning data from startup.
UOS	User Operating System
WARM start	With time and location-assisted information, the end user's current positioning occurs more than 2 to 4 hours after the last positioning.

1.2 Architecture/Process Overview

1.2.1 GNSS Architecture

MT8676 GNSS architecture is as follows, GNSS is in T-box UOS which is also called Yocto UOS:

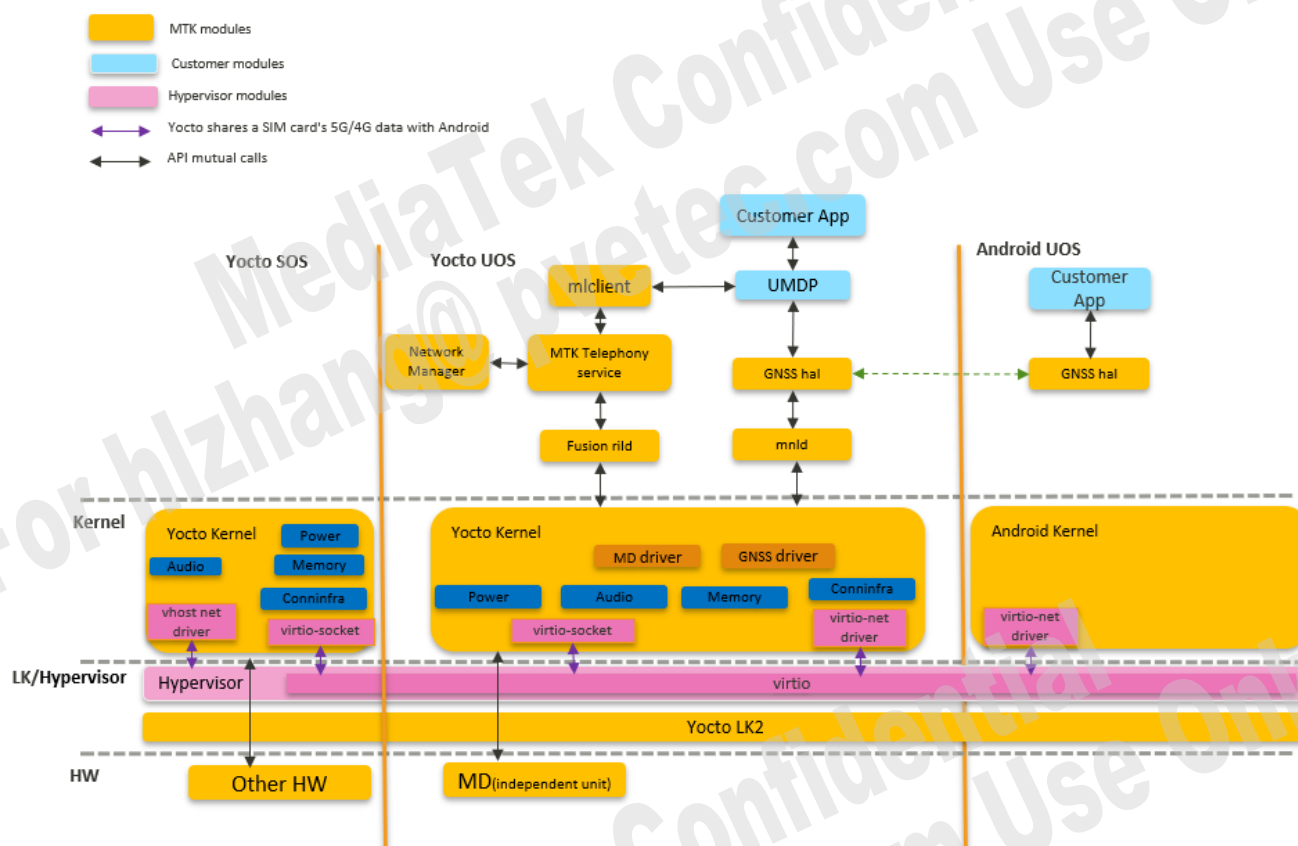


Figure 1-1. GNSS architecture

1.3 Configuration/Customization Guideline

1.3.1 Fixed Rate Configuration

The fixed rate refers to the rate at which GNSS reports location information. Currently, the MT8676 supports fixed rates of: 1Hz, 2Hz, 5Hz, and 10Hz, with a default configuration of 1Hz output. The method to modify the fixed rate is as follows:

Method 1: Modify the code and configure the `fix_interval` parameter. `fix_interval = 100` corresponds to 10Hz; `fix_interval = 1000` corresponds to 1Hz.

Modifications are required in the Yocto environment at

`/src/connectivity/gps/4.0/mtk_mnld/mnld_entity/src/gps_controller.c.`



Figure 1-2. Fixed Rate configuration

Method 2: Dynamically modify the configuration file, take effect after restarting GNSS, modify on the Yocto side.

Command: `echo fix_interval=1000 >> /data/etc/gnss/mnl.prop` /Configured to 1Hz, effective after restarting GPS.

1.3.2 Multi-Satellite Navigation System Configuration

MT8676 supports GPS + GLONASS + Galileo + BeiDou multi-satellite navigation positioning systems. The GNSS operation mode is configured to `MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO_NAVIC`. The default configuration provides optimal GNSS performance. Use the default configuration.

```

885 typedef enum
886 {
887     MTK_CONFIG_GPS_GLONASS = 0,
888     MTK_CONFIG_GPS_BEIDOU,
889     MTK_CONFIG_GPS_GLONASS_BEIDOU,
890     MTK_CONFIG_GPS_ONLY,
891     MTK_CONFIG_BEIDOU_ONLY,
892     MTK_CONFIG_GLONASS_ONLY,
893     MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO,
894     MTK_CONFIG_GPS_GALILEO,
895     MTK_CONFIG_GPS_GLONASS_GALILEO,
896     MTK_CONFIG_GALILEO_ONLY,
897     MTK_CONFIG_GPS_GLONASS_BEIDOU_GALILEO_NAVIC,
898     MTK_CONFIG_GNSS_END
899 } MTK_GNSS_CONFIGURATION;

```

Figure 1-3. GNSS configuration

1.4 GNSS API

1.4.1 Code Path

Code path:

`/src/connectivity/gps/4.0/gps_hal`

1.4.2 GpsInterface

```
typedef struct {
    size_t      size;      // Set sizeof(GpsInterface)
int (*init)(GpsCallbacks_ext* callbacks);      // Initialize callback
    int  (*start)( void );      //gps start
    int  (*stop)( void );      //gps stop
    void (*cleanup)( void );      //cleanup gps

    int  (*inject_time)(GpsUtcTime time, int64_t timeReference,int uncertainty);
// Inject UTC time

    int  (*inject_location)(double latitude, double longitude, float accuracy);
// Inject the current location from another location provider. Latitude and longitude are in
degrees, and accuracy is in meters.

    void (*delete_aiding_data)(GpsAidingData flags);
// Specify that information defined in flags will not be used during the next startup call.
For cold startup, GPS_DELETE_ALL will be passed.

int  (*set_position_mode)(GpsPositionMode mode, GpsPositionRecurrence recurrence,
    uint32_t min_interval, uint32_t preferred_accuracy, uint32_t preferred_time,
    bool lowPowerMode);
// The main purpose is to set the reporting frequency of nmea and location: min_interval

    const void* (*get_extension)(const char* name);
// Obtain a pointer to extension information

    int  (*inject_fused_location)(double latitude, double longitude, float accuracy);
// Inject fused location

    int  (*fix_interval)(uint32_t interval);
    // Dynamically set the reporting frequency, 100: 10Hz, 1000: 1Hz

    int  (*op_mode)(uint8_t opmode);
// Dynamically set operation mode

    int  (*elevation_angle)(uint8_t angle);
// Dynamically set elevation angle constraint

    int  (*l5_disable)(bool disable);
// Dynamically set L5 to enable or disable

    int  (*gps_start_week_num)(uint32_t week);
//Dynamically set GPS start week number
```

1.4.3 Callback

```
typedef struct {
    gps_location_ext_callback location_cb;      // Callback for reporting location; retrieve location
from here, commonly used.
```

```

gps_status_callback status_cb; // GPS status callback, GPS status such as start
and stop
gps_sv_status_callback sv_status_cb; //sv status callback
gps_nmea_callback nmea_cb; // Callback for reporting NMEA data; retrieve NMEA
data from this point, commonly used.
gps_set_capabilities set_capabilities_cb; // Capabilities are used in XTS certification. Obtain
supported capabilities before executing specific test items.
gps_acquire_wakelock acquire_wakelock_cb; // Request callback for wakelock
gps_release_wakelock release_wakelock_cb; // Callback for wakelock status
gps_create_thread create_thread_cb; // Callback for creating a thread
gps_request_utc_time request_utc_time_cb; // Callback for obtaining UTC time
gnss_set_system_info set_system_info_cb; // Callback for setting system information
gnss_sv_status_ext_callback gnss_sv_status_cb; //Obtain the GNSS SV status extension callback

gnss_set_name_callback set_name_cb; //set nmea callback

gnss_request_location_callback request_location_cb; //request location callback
gps_location_ext_callback agps_location_cb; // Obtain AGPS location

gnss_output_rtc3_callback output_rtc3_cb; // Retrieve RTCM callback
gnss_set_signal_type_capabilities_callback set_signal_type_callback; // Set signal type capabilities,
such as carrier frequency.

```

1.4.4 Demo Code

Refer to the usage of `mnld_test` to invoke `GpsInterface` and callback.

`Mnld_test` path: `src/connectivity/gps/4.0/gnss_test`

For example: to call `init` and `fix_interval` in `GpsInterface`

```

GpsInterface_ext* mnld_test_gpsinfs = NULL;
GpsCallbacks_ext* mnld_test_cbs = NULL;
mnld_test_cbs = (GpsCallbacks_ext*)calloc(1, sizeof(GpsCallbacks_ext));
mnld_test_cbs->size = sizeof(GpsCallbacks_ext);
struct gps_device_t_ext *gpsdev = NULL;
gpsdev = &linux_gps_device; // Define variable initialization
mnld_test_gpsinfs = (GpsInterface_ext*)gpsdev->get_gps_interface(gpsdev);
if(mnld_test_gpsinfs != NULL)
{
    LOGI("mnld_test_gpsinfs is not null");
mnld_test_gpsinfs->init(mnld_test_cbs);
    mnld_test_gpsinfs->fix_interval(1000);
}

```

The use of callbacks can also refer to `mnld_test`, `mnld_fm_gps_location_callback`, and `mnld_fm_gps_status_callback`, which are all callback receiving functions, for example:

```

GpsCallbacks_ext mnld_fm_gps_callbacks = {
    .size = sizeof(GpsCallbacks_ext),
    .location_cb = mnld_fm_gps_location_callback,
    .status_cb = mnld_fm_gps_status_callback,
    .....
}

```

Examine the `mnld_test` code closely; it contains specific interface calls and callback usage.

1.5 Frequently Asked Questions/Troubleshooting

1.5.1 Issues Related to Logs

- What logs do MediaTek engineers need for problem analysis?

The Android side requires: /data/debuglogger/mobilelog

T-box UOS (Yocto) side requires: /data/debuglogger/mobilelog, and NMEA log

The T-box UOS (Yocto) mobilelog is enabled by default; to enable the NMEA log, you need to create the /data/etc/gnss/mnl.prop file and write the following in it:

```
debug.dbg2file=1
debug.filename=/data/debuglogger/gpsdebug.log
```

After saving and restarting, the NMEA log will appear in /data/debuglogger/gpsdebug.log.

- How to connect powergps

First, enter the following command:

```
adb shell "echo \"pmtk.serial.port=7000\" > /data/etc/gnss/mnl.prop"
adb shell "echo \"debug_type=0\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.debug_nmea=1\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.dbg2file=1\" >> /data/etc/gnss/mnl.prop"
adb shell "echo \"debug.filename=/data/debuglogger/gpsdebug.log\" >> /data/etc/gnss/mnl.prop"
```

After restarting, enter the following again:

```
adb forward tcp:7000 tcp:7000
```

And use mnld_test to open GPS:

```
adb shell
mnld_test start c &
```

1.5.2 Testing-related Issues

- Before testing, it is necessary to check whether there is satellite signal and whether it is in an open sky environment. Testing the GNSS satellite search or positioning function requires an open sky environment for the signal, such as an open outdoor area or a laboratory with a signal amplifier. The ability to locate is based on the prerequisite that there are more than 6 satellites with a CNR of 40~43 dBm. ---> When measuring GNSS, it is important to pay attention to this; if the current signal environment does not meet the requirements, place a comparison device in the same environment for comparison.
- How to test the TTFF of these startup methods: FULL start, WARM start, COLD start, and HOT start? On the Android side, use YGPS in engineering mode or execute the following adb command to open YGPS. Test using the FULL, COLD, WARM, and HOT buttons: 1. Access engineering mode. 2. Open YGPS using adb command. 3. Select FULL, COLD, WARM, or HOT button for testing.

```
adb shell am start com.mediatek.ygps/.YgpsActivity
```

T-box UOS (Yocto) side can utilize mnld_test.

```
Start test(open gps): mnld_test -h
```

```
sh-3.2# mnld_test -h
mnld_test -h
mnld_test -h
MNLD client test :
-----
The command to start GNSS test:
mnld_test start [start type] [restart times] [restart interval]
[start type]:
h/H: hot start;
w/W: Warm start;
c/C: Cold start;
f/F: Full start
[restart times]: integer value, range is 0-1000, default is 0(no restart, always on).
[restart interval]: integer value, range is 0-3600, the unit is second; the default value is 60
-----
The command to stop GNSS test:
Ctrl+C or kill -9 `pidof mnld_test`
-----
The command to update network status:
mnld_test network [type] [roaming]
[type]: wifi, mobile, disable
[roaming]: roaming, the mobile network is in roaming state
sh-3.2#
```

Figure 1-4. MNLD test

According to the prompt, for example, if you need to test a cold start, you can use the following command:

```
mnld_test start c &
```

Enter the following command to log data to the serial port, which allows viewing of TTFF and other information.

```
journalctl -f --no-tail -o short-precise |grep mnldtest
```

1.5.3 GNSS Path

Source code:

Android: vendor\mediatek\proprietary\hardware\connectivity\gnss

T-box UOS (Yocto): src/connectivity/gps/4.0

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.